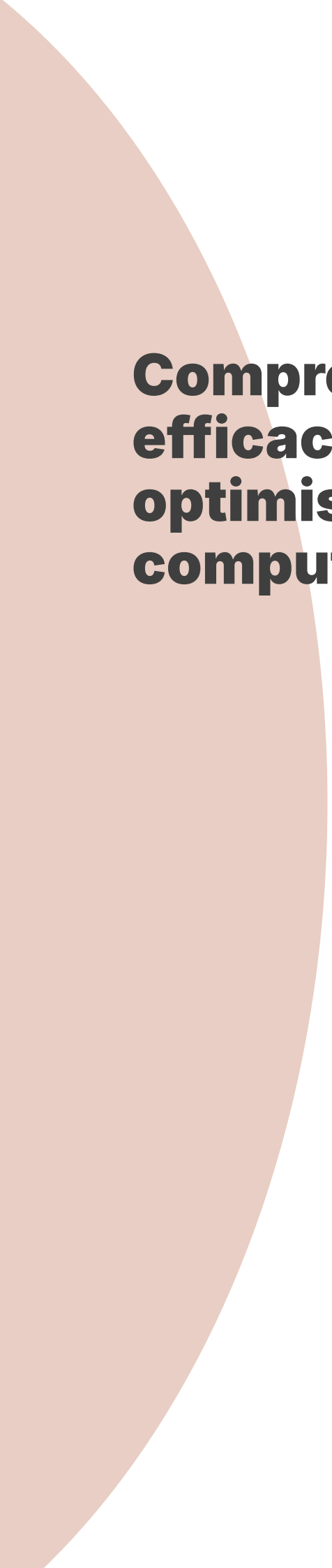




Compression, efficacité et optimisation computationnelle

Table des matières

I - Contexte	3
II - Diagnostic de performance et identification des goulots d'étranglement	4
A. Méthodologie de profilage et identification des goulots d'étranglement.....	4
B. Benchmarking et métriques d'évaluation.....	6
III - Techniques de compression et d'optimisation computationnelle	8
A. Compression et simplification des modèles	8
B. Optimisation du code, du graphe et du pipeline d'inférence	10
IV - Stratégies de déploiement et intégration dans le MLOps	12
A. Déploiement efficace et choix d'architecture	12
B. Industrialisation de l'optimisation et collaboration inter-métiers.....	13
V - Synthèse	15
VI - Auto-évaluation	19
VII - Exercice : Optimiser un modèle de segmentation d'images médicales pour le déploiement hospitalier	24
Solutions des exercices	26

I Contexte

Contexte

Avec la généralisation des modèles d'IA massifs et l'essor des applications déployées – du cloud aux objets embarqués ou mobiles – le défi de l'**efficacité computationnelle** et de la **compression des modèles** prend une importance stratégique. Les réseaux de neurones profonds, en particulier, affichent souvent des millions, voire des milliards de paramètres, nécessitant d'importantes capacités de calcul, de mémoire et d'énergie, limitant leur démocratisation sur des équipements à ressources contraintes ou sur de larges flottes distribuées.

Dans l'industrie, cette question d'**optimisation computationnelle** se pose à toutes les étapes : accélérer l'entraînement pour réduire les coûts, déployer des modèles sur des serveurs avec un faible impact carbone, ou garantir une latence minimale pour embarquer l'IA dans des smartphones, des capteurs connectés, des robots ou des véhicules autonomes.

Par ailleurs, la montée de la **réglementation environnementale** et la nécessité de maîtriser la consommation énergétique amènent les entreprises à repenser la conception et l'exploitation de leurs solutions IA : un modèle performant mais trop lourd peut devenir inutilisable ou non compétitif face à une version plus légère, moins consommatrice mais judicieusement optimisée.

Cette leçon explore les différentes stratégies permettant de **réduire la taille, le coût et la consommation des modèles IA** sans sacrifier, ou en limitant au maximum, la perte de performance : techniques de compression, de distillation, de quantification, d'optimisation logicielle et matérielle. À travers des exemples issus de domaines aussi variés que la santé, la mobilité, la finance ou l'industrie connectée, elle met en lumière que la recherche d'efficacité computationnelle ne concerne plus seulement la « *technique* », mais devient un véritable facteur d'innovation, de compétitivité et de durabilité des systèmes IA modernes.

II Diagnostic de performance et identification des goulots d'étranglement

A. Méthodologie de profilage et identification des goulots d'étranglement

Cadre d'analyse des performances computationnelles

L'optimisation computationnelle d'un modèle IA démarre invariablement par un **diagnostic rigoureux de sa performance réelle**. Il ne s'agit pas uniquement d'observer un score de précision, mais d'analyser de façon multidimensionnelle : temps d'inférence (latence), utilisation mémoire, consommation CPU/GPU, occupation I/O, débit maximal, taux d'utilisation du batch processing, etc. Ces métriques sont à relever sur différentes plateformes (machine locale, cloud, edge device) et pendant les phases clefs (prétraitement, inférence, post-traitement).

Dans un contexte industriel, la pertinence des métriques varie selon l'environnement cible. Sur le cloud, le coût et le débit priment souvent : il s'agit de délivrer le plus de prédictions possibles par euro dépensé et d'optimiser la scalabilité horizontale. Pour les appareils edge ou mobiles, les contraintes de mémoire, d'autonomie énergétique et de latence deviennent déterminantes : un modèle trop volumineux ne peut tout simplement pas être embarqué.

Établir un tel cadre requiert aussi de contextualiser les objectifs : une application d'aide au diagnostic médical peut exiger une latence <100ms, alors qu'un scoring bancaire tolérera quelques secondes d'attente mais interdira toute sortie de RAM en raison de la confidentialité. Ce diagnostic préalable oriente la stratégie d'optimisation vers les véritables facteurs limitants, et non vers une simple chasse à la microseconde superficielle.

Diagnostic appliqué à un assistant vocal embarqué

[👁 Exemple](#)

Une startup souhaitait embarquer son modèle de transcription sur des casques Bluetooth. Le premier diagnostic de latence a révélé que le temps d'inférence était multiplié par 10 sur la puce ARM par rapport à la station de développement, en raison du débit limité de la RAM et de la taille initiale du modèle. Les priorités d'optimisation ont ainsi été redéfinies autour de la compression mémoire et de la gestion des accès disques.

Latence

[Az Définition](#)

La **latence** correspond au temps écoulé entre la réception d'une donnée en entrée et la production d'une prédiction par le modèle IA. Cette notion est centrale dans toute application interactive, temps réel, ou embarquée, où la réactivité est aussi décisive que la précision.

Techniques de profilage avancé

Le **profilage** ne se limite pas à l'observation générale des temps de réponse : il s'agit de cartographier l'ensemble du pipeline pour distinguer précisément les séquences les plus consommatrices de ressources. À ce titre, des outils comme PyTorch Profiler, TensorFlow Profiler et NVIDIA Nsight sont devenus incontournables. Ils permettent d'analyser les opérations ligne à ligne, de visualiser la charge sur chaque composant matériel (CPU, GPU, RAM) et d'identifier les tâches qui « *bloquent* » le pipeline, empêchant toute parallélisation.

La visualisation graphique (heatmap ou graphe d'exécution) rend possible l'identification des nœuds critiques où la totalité de la chaîne attend la fin d'une opération lourde (ex : convolution 2D sur entrée géante, data loader séquentiel sur disque lent). On distingue alors les séquences que l'on peut paralléliser, vectoriser ou fusionner, des sections qui doivent impérativement être optimisées ou repensées (découpage, réécriture, changement d'algorithme, adaptation à l'infra matérielle).

Enfin, le profilage sert aussi à comparer différentes versions du même modèle, évaluer l'impact d'une mise à jour de code ou d'une migration de plateforme, et justifier technique et économiquement la sélection d'un compromis performance/complexité donné.

Heatmap de profilage d'un modèle IA



Ne pas négliger la configuration matérielle

 **Attention**

Profiler un modèle IA sur une machine de développement ultra-puissante peut masquer des goulots d'étranglement qui apparaîtront immédiatement dans l'environnement cible de production. Toujours profiler en condition « réaliste » ou simuler la configuration finale pour anticiper les écarts.

Analyse mémoire et transferts de données

L'**occupation mémoire** et les transferts de données constituent souvent le premier facteur limitant sur les infrastructures réelles. Cela concerne à la fois les échanges entre composants locaux (RAM et CPU/GPU) et les flux distants (stockage réseau, cloud, edge). Un mauvais alignement batch size et capacité mémoire, des formats d'entrée trop volumineux ou une gestion sous-optimale du cache peuvent saturer le système, générant de la latence cachée et des ralentissements peu visibles côté code.

Identifier ces points de charge nécessite de croiser les lectures de profiling avec des outils dédiés à l'analyse mémoire : suivi de la VRAM sur GPU, inspection de la RAM système, mesure de l'usage cache/disk, monitoring réseaux.

Dans certains cas, la restructuration du prétraitement, le changement de format d'entrée (TFRecord, Parquet, etc.) ou la modification du pipeline (préchargement asynchrone, découpage) suffisent à diviser par deux voire trois la latence perçue sans toucher au modèle lui-même.

Bande passante mémoire

Az Définition

La **bande passante mémoire** mesure la quantité maximale de données qui peut être transférée entre la mémoire (RAM, VRAM, cache) et les unités de calcul. Si cette bande passante est saturée, aucune optimisation logicielle ne peut compenser ce goulet d'étranglement matériel.

Techniques avancées de profilage pour les modèles d'IA

B. Benchmarking et métriques d'évaluation

Établir une baseline de performance fiable

Toute démarche d'optimisation nécessite de partir d'un **point de référence robuste** – une baseline – pour mesurer objectivement l'apport des modifications. Définir cette baseline, c'est créer des scénarios de test représentatifs : ils doivent reproduire les conditions réelles ou cibles du déploiement (hardware, volume, environnement concurrent, type de requêtes).

Des métriques adaptées (latence, débit, consommation mémoire, efficacité énergétique, etc.) sont alors collectées de façon systématique, et chaque évolution du code, du modèle ou du pipeline s'évalue à l'aune de ce référentiel.

La baseline devient vite, dans les équipes matures, un outil collectif : elle sert à documenter les progrès, alerter sur les régressions, et arbitrer entre différentes propositions d'optimisation. Des benchmarks réalistes évitent de s'illusionner sur des performances « *en laboratoire* » qui s'effondreraient en production réelle.

Par ailleurs, la baseline doit être réactualisée à chaque évolution majeure de l'environnement cible (ajout d'une nouvelle plateforme, migration cloud, changement de prétraitement), sous peine de perdre tout le bénéfice du suivi qualité.

Baseline

Az Définition

La **baseline** désigne l'ensemble des mesures de référence (métriques clés) établies par une équipe IA sur un modèle ou un pipeline dans un contexte donné. Elle permet d'évaluer objectivement les gains ou les pertes d'une optimisation.

Adaptation des métriques au contexte opérationnel

Il n'existe pas de « *bonne métrique* » universelle : le choix des indicateurs de performance dépend d'abord de l'environnement et des contraintes propres au cas d'usage. Pour une application mobile d'imagerie médicale, la consommation mémoire prime ; pour un moteur de recommandation cloud, le débit et la capacité de montée en charge ; pour une caméra intelligente, la latence inframilliseconde ; dans l'industrie, la robustesse de l'inférence sous contrainte de température ou de pertes réseau est cruciale.

Cette adaptation du suivi est critique pour éviter d'optimiser « à côté du réel ». Elle permet également de justifier auprès des métiers les décisions techniques : il est parfois préférable d'accepter une légère baisse de précision si celle-ci garantit une exécution stable sur tous les appareils ou une réduction massive des coûts d'infrastructure.

Bien paramétrées, ces métriques guident non seulement le développement, mais aussi le choix de déploiement (cloud, edge, hybride, etc.) et l'investissement dans le matériel cible.

Débit (throughput)

Az Définition

Le **débit** (throughput) mesure le nombre d'échantillons/processus pouvant être traités ou prédits par unité de temps. C'est une métrique clé pour dimensionner le hardware et garantir la scalabilité, notamment dans les systèmes cloud ou de streaming.

Toujours accompagner les métriques de seuils métier

Conseil

Associer les indicateurs (latence, débit, etc.) à des seuils validés avec les utilisateurs ou opérationnels rend les actions d'optimisation plus efficaces : on sait « *pourquoi* » on agit et « *jusqu'où* » il est pertinent de le faire.

Comparaison multi-environnements

Un modèle IA possède fréquemment des profils de performance très différents selon le support d'exécution : GPU haut de gamme, CPU standard, edge device, cloud partagé, etc. Un benchmarking qui ne prend en compte que le « *meilleur cas* » masque les écueils concrets du déploiement, surtout dans des architectures hybrides où les modèles migrent selon la charge ou la localisation des utilisateurs.

Tester le modèle dans différents environnements, automatiser la collecte, puis comparer l'évolution des métriques sur chaque plateforme permet d'anticiper les besoins d'adaptation – quantification, distillation, choix de batch size – et d'éviter des réécritures de dernière minute souvent coûteuses.

Des entreprises françaises du secteur industriel ou bancaire ont fortement réduit leurs incidents de production en mettant en place des batteries de tests multi-soutiens bien avant le déploiement final.

Ne jamais se fier à un unique test

Attention

Même un modèle « *parfait* » lors des tests unitaires peut révéler des goulots d'étranglement cachés lors des premiers déploiements à l'échelle ou sur du hardware moins performant.

Choisir les bonnes métriques de performance

III Techniques de compression et d'optimisation computationnelle

A. Compression et simplification des modèles

Réduction de la complexité grâce à la distillation et au pruning

La **distillation de connaissances** et le **pruning** sont deux méthodologies majeures pour comprimer un réseau neuronal trop volumineux tout en préservant un maximum de ses propriétés prédictives.

La distillation consiste à entraîner un modèle « élève » (student), beaucoup plus compact, à imiter les prédictions d'un modèle « professeur » (teacher) plus massif. La cible de l'élève n'est alors plus les simples étiquettes (« *hard targets* ») mais la sortie probabiliste du teacher (« *soft targets* »), capturant une partie de la logique fine acquise par le réseau initial. Cette approche permet de transférer efficacité et robustesse, tout en réduisant drastiquement la taille et le coût du modèle résultant.

Le pruning, quant à lui, repose sur l'identification puis la suppression des poids ou neurones les moins influents pour le modèle. On distingue le **pruning non structuré** (suppression arbitraire de poids proches de zéro) et le **pruning structuré** (retrait de couches/d'entités entières : canaux, neurones, têtes d'attention). L'intérêt du pruning ne se limite pas au gain mémoire : il améliore parfois la généralisation, et, combiné à des ré-entraînements successifs, permet de stabiliser la précision tout en allégeant l'empreinte computationnelle.

De nombreuses entreprises spécialisées en vision par ordinateur utilisent aujourd'hui cette combinaison pour déployer des réseaux « *lite* » sur mobile, caméras intelligentes ou robots, sans sacrifier la robustesse sur les cas réels.

Un atout majeur de ces méthodes : elles sont « *universelles* » et peuvent s'appliquer, avec adaptations, à la plupart des types de réseaux (CNN, transformers, RNN, etc.), dès lors que la cible de performance est bien définie et que l'on accepte un léger compromis sur l'exactitude brute au profit d'une efficacité décuplée.

Toujours conserver un modèle de référence



Avant compression (pruning/distillation), archivez le modèle d'origine « *teacher* » et conservez un pipeline d'évaluation complet. Cela permet de comparer aisément l'impact de chaque compression et de revenir en arrière si le modèle compact s'avère trop dégradé.

Risques d'un pruning mal contrôlé



Une suppression excessive ou mal calibrée de poids essentiels provoque un effondrement des performances, des biais ou une évolution imprévue de la sensibilité aux entrées rares. Toujours monitorer la perte de précision sur l'ensemble des sous-populations critiques et non sur la seule moyenne globale.

Pruning non structuré vs structuré

Az Définition

Le **pruning non structuré** consiste à retirer individuellement des poids jugés négligeables, indépendamment de leur position dans la couche, tandis que le **pruning structuré** supprime des blocs entiers (filtres, neurones, têtes) ce qui permet des gains plus significatifs lors du déploiement matériel.

Compression des modèles par distillation et pruning Conception de modèles légers et architectures optimisées

Plutôt que de compresser un modèle lourd après-coup, de nombreuses architectures sont construites d'emblée pour l'**efficacité**. Des familles comme MobileNet, Efficient Net, ShuffleNet illustrent ce mouvement, en intégrant des blocs factorisés (convolutions groupées, depthwise séparable, squeeze-and-excitation), des mécanismes d'attention légère, ou une échelle optimisée profondeur/largeur selon les besoins.

Cette conception « *compacte* » permet de conserver une part importante de performance en cible tout en maîtrisant la consommation mémoire, l'énergie, et la compatibilité avec des hardwares très divers. L'approche consiste aussi à sélectionner intelligemment la résolution d'entrée, la largeur des couches, la structures des blocs, etc.

Une société française spécialisée en NLP a utilisé une telle architecture pour développer un assistant vocal multilingue capable de tourner en local sur smartphone d'entrée de gamme : grâce à la compacité du modèle et à la préparation spécifique pour ARM, le gain en autonomie et en réactivité a dépassé 70 % sans chute de précision sur l'ensemble des langues gérées.

MobileNet en vision industrielle

👁 Exemple

Dans une usine d'embouteillage, le passage d'un modèle classique CNN à une architecture MobileNet réduite a permis d'embarquer le contrôle qualité sur une caméra connectée, tout en divisant par 15 la consommation d'énergie et en conservant une précision >98 %.

Anticiper les contraintes hardware

👤 Conseil

Avant d'arrêter une architecture légère, établir un « *profil hardware* » cible : RAM disponible, unités SIMD présentes, spécificités GPU/CPU/FPGA, besoins d'exécution offline ; ce diagnostic conditionne le choix du type de modèle compact.

Techniques de quantification pour une exécution plus rapide

Au-delà de la compression structurelle, la **quantification** vise à réduire la précision numérique des poids et des activations des modèles de deep learning. Passer des poids flottants 32bits à INT8, INT4, voire au binaire, peut accélérer massivement l'inférence tout en divisant la consommation mémoire.

La quantification post-entraînement consiste à ré-encoder le modèle existant, généralement après calibration sur un set de validation, alors que la quantification-aware training implique d'apprendre dès l'origine avec des contraintes de représentation réduite, ce qui donne souvent une meilleure résilience à la chute de précision.

L'impact varie selon l'architecture : des réseaux de convolution ou MLP se prêtent très bien à l'opération, alors que certains transformers ou réseaux à attention y sont plus sensibles. Dans l'industrie, un acteur de la vidéosurveillance a obtenu des gains spectaculaires en deployment edge (caméras intelligentes) sans compromettre la qualité de détection, grâce à une double quantification des poids et des activations.

INT8/INT4

Az Définition

Les types **INT8** (entiers codés sur 8 bits) et **INT4** (4 bits) sont des précisions numériques réduites permettant de stocker et traiter plus de valeurs avec moins de mémoire, au prix d'une approximation contrôlée des calculs.

B. Optimisation du code, du graphe et du pipeline d'inférence

Fusion d'opérations et graphes optimisés pour l'exécution

L'optimisation d'un modèle ne se joue pas seulement sur sa structure globale, mais sur la manière dont il est exprimé sous forme de graphe computationnel. La **fusion d'opérations** consiste à regrouper plusieurs calculs successifs (par exemple : convolution puis activation puis batch normalization) en une seule opération optimisée, limitant ainsi les allers-retours en mémoire et les surcoûts liés aux appels séparés.

Cette optimisation réduit la latence, la consommation mémoire, et simplifie la chaîne d'exécution pour les frameworks de calcul. Restructurer ainsi le graphe permet également de tirer parti des optimisations matérielles ou des possibilités avancées du compilateur cible (vectorisation, exécution pipeline, offloading automatique, etc.).

Une entreprise de santé a ainsi accéléré de plus de 40 % son modèle de segmentation d'images médicales, simplement en adoptant une version du graphe où les séquences lourdes étaient fusionnées et où les dépendances inutiles avaient été supprimées. Cette approche a permis une exécution sans interruption (pipeline) sur GPU, sans compromis sur la précision diagnostique.

Appuyez-vous sur les outils open source

 Conseil

Les frameworks comme ONNX, TensorRT ou TVM sont gratuits, documentés, et soutenus par de grandes communautés. Leur adoption permet d'accéder rapidement à des gains de performance sans investir dans des développements maison risqués.

Optimisation via compilateurs spécialisés (TVM, TensorRT, XLA)

Az Définition

Les **compilateurs spécialisés** jouent aujourd'hui un rôle clé dans l'optimisation fine des modèles IA pour chaque environnement matériel. TVM, TensorRT, XLA et d'autres frameworks fournissent des outils pour transformer un graphe haut niveau (TensorFlow, PyTorch) en une version ultra-optimisée (suppression de redondances, réordonnancement des opérations, allocation mémoire customisée).

Leur intérêt : offrir des gains de performance parfois spectaculaires (jusqu'à 10x sur la latence ou l'énergie), tout en générant des exécutables adaptés à des matériels très diversifiées (GPU Nvidia, CPU, NPU, FPGA).

Une startup française opérant en production cloud a par exemple divisé par 5 son coût unitaire d'inférence en convertissant ses modèles via TVM, avec adaptation automatique sur plusieurs versions de GPU et CPU cibles.

Toujours mesurer le trade-off précision/performance après compilation

 Conseil

Certaines optimisations profondes (fusion, inlining agressif, quantification automatique) peuvent induire une légère perte de précision. Il est indispensable de re-valider le modèle sur l'ensemble des jeux d'évaluation après chaque changement compilateur/hardware.

Optimisation du pipeline d'inférence et prétraitement des données

Les gains d'efficacité ne viennent pas uniquement de l'architecture ou du code modèle, mais dépendent fortement de la fluidité du **pipeline d'inférence** dans son ensemble.

L'utilisation du préchargement asynchrone permet d'entraîner ou inférer sans temps mort d'alimentation, la parallélisation du prétraitement répartit la charge sur plusieurs cœurs, la mise en cache évite de recalculer des étapes identiques pour chaque lot, et l'adoption de formats optimisés (TFRecord, Parquet) accélère la lecture disque-mémoire.

Une grande plateforme d'e-commerce a doublé la réactivité de son moteur de recommandation rien qu'en adaptant son pipeline afin d'exploiter le cache, découper les jobs et paralléliser la normalisation sur chaque nœud.

Surveiller les goulets d'étranglement « hors modèle »

 Attention

Dans de très nombreux cas, l'origine des ralentissements se situe dans l'accès aux données (I/O, prétraitement) et non dans le modèle. Un pipeline non optimisé peut réduire à néant les efforts d'accélération du modèle.

Stratégies d'exécution adaptative selon les contraintes

L'**exécution adaptative** consiste à ajuster dynamiquement la complexité ou les paramètres du modèle selon l'état des ressources disponibles, la charge du serveur, ou la nature des données à traiter. Selon la pression RAM ou CPU, on peut réduire à la volée le batch size, choisir un sous-modèle plus léger (cascade), ou activer seulement certaines branches du réseau selon la complexité (conditional computation). Cette agilité permet d'assurer la robustesse du service : un modèle qui allège temporairement sa charge évite l'échec ou le timeout qui compromet la fiabilité globale.

Un exemple marquant est celui d'une application de traduction instantanée embarquée : sur mobile haut de gamme, le modèle complet est activé ; sur appareil modeste, une version réduite est sélectionnée automatiquement, assurant à l'utilisateur une expérience fluide quel que soit l'environnement.

Concevoir pour la dégradation « élégante »

 Conseil

Toujours prévoir, dès l'amont, les cas où le modèle devra « *s'amincir* » pour tenir la charge : architecture en sous-modèles, configuration dynamique, ou fallback explicites permettent d'offrir un service continu même sous contrainte extrême.

IV Stratégies de déploiement et intégration dans le MLOps

A. Déploiement efficace et choix d'architecture

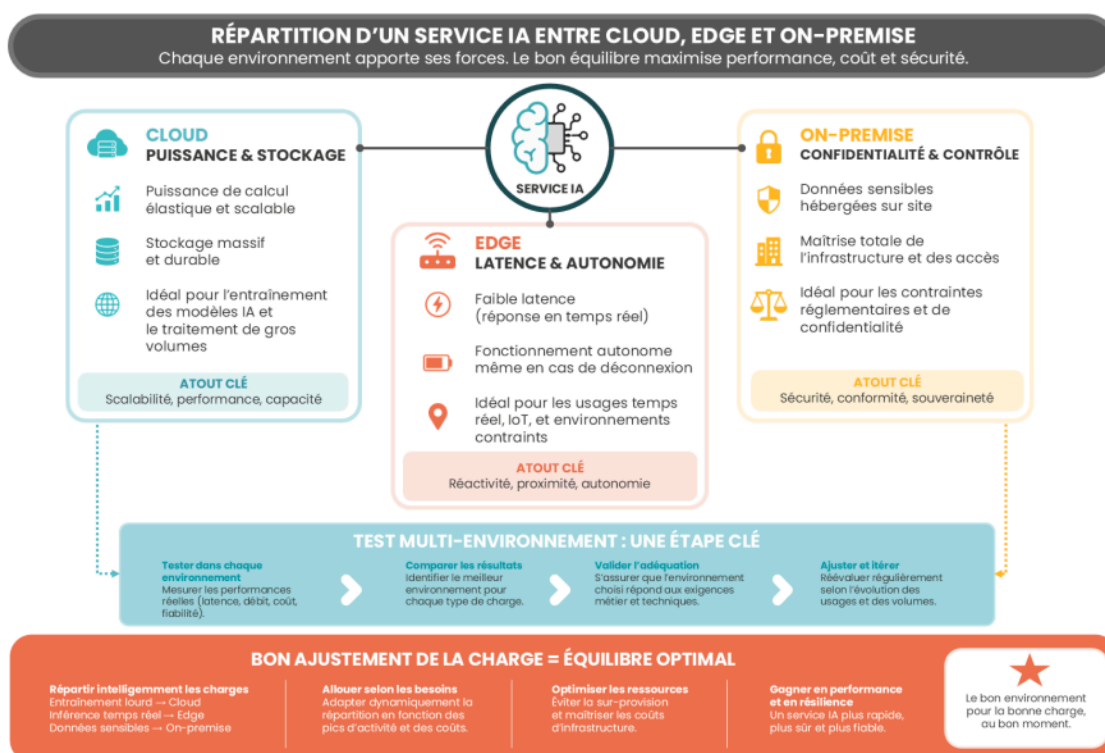
Choisir l'infrastructure optimale selon le contexte de déploiement

La sélection de l'environnement de déploiement pour un modèle IA est une étape aussi stratégique que la conception du modèle lui-même. Le choix dépend de nombreux critères : la **latence attendue** (exigence temps réel ? asynchrone ?), les **contraintes budgétaires** (capacité cloud, coût du hardware embarqué, mutualisation possible), les **ressources matérielles** disponibles (GPU, CPU, RAM, stockage) ainsi que les exigences de confidentialité, de souveraineté ou de conformité réglementaire.

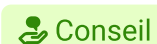
Différents cas d'usage appellent des infrastructures radicalement différentes. Pour le diagnostic médical en hôpital, l'impératif de confidentialité, le besoin de latence faible et la volumétrie de données imposent souvent un déploiement local sur des serveurs disposant de puissants GPU. À l'inverse, la reconnaissance d'image embarquée pour la mobilité autonome privilégie des modèles allégés, capables de tourner sur des edge devices parfois dépourvus de connexion stable, avec de strictes limites d'énergie.

Un processus méthodique doit donc évaluer les charges réelles, comparer les scénarios d'usage (cloud, edge, on-premise, hybride), effectuer des POC sur cibles variées et valider la robustesse dans la configuration la plus contrainte.

Schéma cloud/hybride/edge



Adapter l'architecture logicielle pour maximiser l'efficacité



La réussite du déploiement dépend, au-delà du choix matériel, de l'architecture logicielle : une structure adaptée permet de configurer efficacité, robustesse et facilité de maintenance. Les **microservices** offrent modularité et scalabilité mais complexifient la gestion des

dépendances ; les modèles batchés réduisent les coûts mais augmentent la latence pour certains utilisateurs ; architectures orientées événements ou files de messages (message broker) apportent flexibilité, mais imposent une orchestration soignée.

L'expérience d'une entreprise de cybersécurité française, qui a réduit la latence de ses modèles de détection de menaces de 70 % en optant pour une communication asynchrone entre IA temps réel et applications métier, illustre la puissance d'un bon choix architectural sur la performance globale.

Les architectures « parfaites » n'existent pas

⚠ Attention

Chaque organisation doit adapter ses choix aux contraintes métier, à la maturité de ses équipes et à la criticité du service fourni. Les compromis entre modularité, performance, coût et sécurité sont inévitables et doivent être discutés collectivement.

Assurer la portabilité et la résilience du modèle en production

Un déploiement industriel ne peut s'affranchir de la **portabilité** (capacité à migrer facilement entre environnements) et de la **résilience** (capacité à résister aux erreurs, crashes, mises à jour). Des outils comme **Docker** (containerisation), **Kubernetes** (orchestration), ou les gestionnaires de versions de modèles (**MLflow**, DVC) offrent des solutions concrètes pour automatiser les déploiements, gérer les rollback, orchestrer la montée en charge et sécuriser le monitoring.

Dans l'industrie aéronautique, la mise en place d'une architecture orchestrée, capable de basculer le modèle d'un cloud public à une infrastructure interne sans rupture de service, a permis d'assurer continuité opérationnelle et respect des exigences contractuelles complexes (multi-sites, redondance, traçabilité).

B. Industrialisation de l'optimisation et collaboration inter-métiers

Automatiser l'optimisation dans un cycle MLOps

L'intégration de l'optimisation computationnelle dans les pipelines de **développement continu** (CI/CD) transforme la performance IA en un processus automatisé, reproductible et contrôlable. Les étapes de profiling, la comparaison systématique de versions compressées ou quantifiées, ainsi que la sélection automatisée du meilleur compromis performance/précision, deviennent des maillons naturels de la chaîne.

Le cas d'une entreprise de robotique est exemplaire : grâce à ONNX Runtime intégré dans son workflow CI/CD, chaque « *push* » de nouveau modèle génère automatiquement des versions optimisées, testées selon les critères métier, avec génération d'alertes en cas de régression de performance ou de consommation excessive. Ce fonctionnement industrialisé accélère le time to market et sécurise la fiabilité en production.

CI/CD pour MLOps

Az Définition

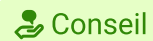
Le cycle **CI/CD** (« *Continuous Integration / Continuous Delivery* ») automatisé appliqué à l'IA intègre le profiling, l'optimisation, le test métrique et le déploiement, garantissant à chaque itération que le modèle reste performant, fiable et répond aux contraintes terrain.

Favoriser la collaboration entre experts métier, data scientists et ingénieurs

L'**optimisation computationnelle durable** dépend d'une interaction forte entre tous les métiers : experts métier qui précisent les contraintes d'usage, data scientists qui adaptent les modèles et ingénieurs qui déploient et maintiennent les infrastructures. Formaliser ces échanges (via des workflows partagés, des « tickets » de spécification, des revues croisées) accélère la résolution des problèmes complexes et assure une optimisation réellement alignée sur les besoins opérationnels.

L'exemple d'un service public français ayant instauré des « *ateliers communs d'optimisation* » a permis de diviser par deux les temps de mise en production tout en améliorant la satisfaction des utilisateurs métiers.

Capitaliser sur l'expérience collective



Conseil

Documentez systématiquement les retours terrain, les « recettes » d'optimisation réussie et les difficultés rencontrées : cette capitalisation évite les pertes de savoir lors des départs, rotations ou changements d'équipe projet.

Structurer la capitalisation et la veille technologique

La construction de **référentiels internes** (documentation, tableaux de bord de performance par environnement, retours d'expérience compilés, comparatifs de versions, veille technologique) s'avère clé pour :

- Éviter de répéter des processus de « *trial & error* » coûteux.
- Accélérer la prise de décision sur les choix d'optimisation.
- Faciliter la formation des nouveaux arrivants.

Un grand groupe industriel a ainsi normalisé ses démarches d'audit d'efficacité grâce à un tableau de bord centralisé de performances modèles : cela a permis, en quelques trimestres, de mieux suivre l'évolution, d'éviter les régressions et de piloter la veille active sur les solutions logicielles et matérielles.

V Synthèse

A. Flashcards

Latence

Temps écoulé entre réception d'une donnée et production d'une prédiction par le modèle IA.

Profilage

Analyse détaillée du pipeline pour identifier les opérations consommatrices de ressources computationnelles.

Baseline

Ensemble de mesures de référence permettant d'évaluer objectivement les gains d'une optimisation.

Distillation de connaissances

Entraînement d'un modèle compact à imiter les prédictions d'un modèle massif pour transférer performance.

Pruning structuré

Suppression de blocs entiers (filtres, neurones, têtes) pour gains matériels significatifs au déploiement.

Quantification

Réduction de la précision numérique des poids et activations pour accélérer l'inférence et réduire mémoire.

Fusion d'opérations

Regroupement de calculs successifs en une opération unique pour limiter transferts mémoire et latence.

TensorRT

Compilateur spécialisé NVIDIA optimisant graphes IA pour exécution ultra-rapide sur GPU.

Débit (throughput)

Nombre d'échantillons traités par unité de temps, métrique clé pour scalabilité et dimensionnement.

MobileNet

Architecture légère conçue pour efficience sur appareils mobiles via convolutions depthwise séparables.

Exécution adaptative

Ajustement dynamique de la complexité du modèle selon ressources disponibles ou charge système.

MLOps

Pratiques d'industrialisation IA intégrant développement, déploiement, monitoring et optimisation continue.

Containerisation

Encapsulation d'un modèle et de ses dépendances dans un conteneur pour portabilité et reproductibilité.

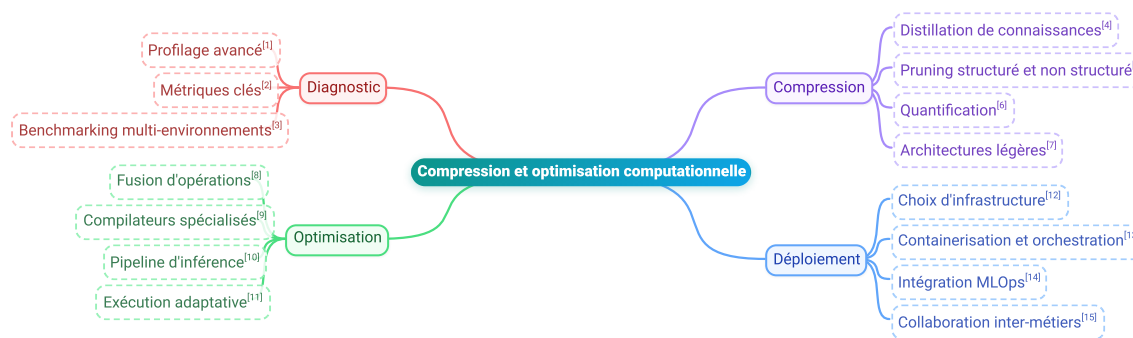
Bande passante mémoire

Quantité maximale de données transférables entre mémoire et unités de calcul par unité de temps.

La théorie en Bref

🔗 Fondamental

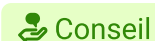
La **compression** et l'**optimisation computationnelle** visent à réduire la taille, le coût et la consommation énergétique des modèles IA sans dégrader significativement leurs performances. Le **profilage** identifie les **goulots d'étranglement** (latence, mémoire, I/O) pour orienter les actions d'optimisation. Les techniques majeures incluent la **distillation de connaissances** (transfert d'un modèle lourd vers un modèle compact), le **pruning** (suppression de poids ou neurones peu influents), la **quantification** (réduction de la précision numérique des poids) et l'adoption d'**architectures légères** (MobileNet, EfficientNet). L'optimisation du **graphe computationnel** (fusion d'opérations, compilateurs spécialisés comme TensorRT ou TVM) et du **pipeline d'inférence** (préchargement asynchrone, parallélisation) complètent la démarche. Le déploiement efficace repose sur le choix d'une **infrastructure adaptée** (cloud, edge, hybride), la **containerisation** (Docker, Kubernetes) et l'intégration dans un cycle **MLOps** automatisé garantissant performance, portabilité et résilience en production.



1. Identification des goulots d'étranglement via outils spécialisés (PyTorch Profiler, TensorFlow Profiler, NVIDIA Nsight).
2. Latence, débit, consommation mémoire, efficacité énergétique selon le contexte opérationnel.
3. Comparaison des performances sur cloud, edge, GPU, CPU pour anticiper les écarts de déploiement.
4. Transfert des capacités d'un modèle lourd (teacher) vers un modèle compact (student).
5. Suppression de poids, neurones ou couches peu influents pour alléger le modèle.
6. Réduction de la précision numérique (FP32 vers INT8/INT4) pour accélérer l'inférence.
7. Conception native pour l'efficacité (MobileNet, EfficientNet, ShuffleNet).
8. Regroupement de calculs successifs pour limiter les transferts mémoire.
9. TensorRT, TVM, XLA pour optimisation fine du graphe computationnel.
10. Préchargement asynchrone, parallélisation, mise en cache, formats optimisés.
11. Ajustement dynamique de la complexité selon les ressources disponibles.
12. Cloud, edge, on-premise ou hybride selon latence, budget, confidentialité.
13. Docker, Kubernetes pour portabilité, résilience et montée en charge.
14. Automatisation du profilage, tests multi-plateformes, monitoring continu dans le cycle CI/CD.
15. Alignement experts métier, data scientists et ingénieurs pour optimisation durable.

Compression et optimisation computationnelle

Les conseils pro



Conseil

Comment identifier efficacement les goulots d'étranglement d'un modèle IA en production ?

- Profiler le modèle dans l'environnement cible réel, pas uniquement sur machine de développement.
- Utiliser des outils spécialisés (PyTorch Profiler, TensorFlow Profiler, NVIDIA Nsight) pour cartographier chaque opération.
- Mesurer latence, débit, consommation mémoire et I/O sur plusieurs scénarios d'usage représentatifs.
- Comparer les performances sur différentes plateformes (cloud, edge, GPU, CPU) pour anticiper les écarts de déploiement.

Comment choisir entre distillation, pruning et quantification pour compresser un modèle ?

- Évaluer la tolérance métier à la perte de précision et fixer un seuil de dégradation acceptable.
- Privilégier la distillation si vous disposez d'un modèle teacher performant et de ressources d'entraînement.
- Opter pour le pruning structuré si l'objectif est un gain matériel immédiat et une simplification du graphe.
- Appliquer la quantification post-entraînement pour un déploiement rapide sur edge avec gains mémoire et vitesse.

Comment intégrer l'optimisation computationnelle dans un pipeline MLOps existant ?

- Automatiser le profilage et le benchmarking à chaque itération du cycle CI/CD.
- Définir des seuils de performance (latence, débit, mémoire) comme critères de validation avant déploiement.
- Générer automatiquement des versions optimisées (quantifiées, compilées) et les tester en parallèle.
- Documenter chaque optimisation appliquée et capitaliser les retours d'expérience dans un référentiel partagé.

Comment garantir la portabilité et la résilience d'un modèle IA déployé à grande échelle ?

- Containeriser le modèle et ses dépendances avec Docker pour assurer reproductibilité sur tout environnement.
- Orchestrer le déploiement avec Kubernetes pour gérer montée en charge, rollback et haute disponibilité.
- Tester le modèle sur toutes les plateformes cibles avant mise en production pour détecter les régressions.
- Mettre en place un monitoring continu des métriques de performance et déclencher des alertes en cas de dérive.

Comment arbitrer entre performance, coût et impact environnemental lors du déploiement d'un modèle IA ?

- Mesurer la consommation énergétique réelle du modèle sur l'infrastructure cible et la rapporter au coût opérationnel.
- Comparer plusieurs scénarios de déploiement (cloud mutualisé, edge local, hybride) en intégrant critères RSE.
- Privilégier les architectures légères et les optimisations compilateur pour réduire empreinte carbone sans sacrifier qualité.

- Documenter les arbitrages et les communiquer aux parties prenantes pour aligner décisions techniques et engagements stratégiques.

VI Auto-évaluation

Exercice 1 : Diagnostic et profilage des performances

[solution n°1 p. 26]

Exercice

Profilier un modèle IA exclusivement sur une machine de développement haute performance est suffisant pour anticiper les performances en production.

Vrai

Faux

Exercice

Vous déployez un modèle IA sur un capteur connecté embarqué, sans connexion réseau stable. Quelle combinaison de métriques doit primer dans votre démarche de benchmarking ?

Débit maximal et scalabilité horizontale

Latence, consommation mémoire et autonomie énergétique

Précision globale et coût d'inférence par requête

Taux d'utilisation du batch processing et occupation I/O réseau

Exercice

Associez chaque outil ou concept de profilage à sa description fonctionnelle principale.

A Baseline

B Heatmap ou graphe d'exécution

C PyTorch Profiler / TensorFlow Profiler

D Bande passante mémoire

Analyse ligne à ligne des opérations et visualisation de la charge sur chaque composant matériel	Représentation visuelle permettant d'identifier les nœuds critiques qui bloquent la parallélisation	Quantité maximale de données transférable entre mémoire et unités de calcul, dont la saturation crée un goulot matériel incompressible	Ensemble de mesures de référence permettant d'évaluer objectivement les gains ou pertes d'une optimisation
--	---	--	--

Exercice 5 : Techniques de compression des modèles

[solution n°2 p. 27]

Exercice

Le pruning d'un réseau neuronal dégrade systématiquement sa capacité de généralisation.

☐ Vrai☐ Faux**Exercice**

Dans une démarche de distillation de connaissances, qu'est-ce que le modèle « élève » (student) apprend à reproduire en priorité ?

☐ Les étiquettes binaires exactes (hard targets) issues du jeu d'entraînement d'origine☐ Les sorties probabilistes du modèle teacher (soft targets), qui capturent la logique fine du réseau massif☐ Les poids internes du modèle teacher, couche par couche☐ La structure architecturale complète du modèle teacher, réduite par facteur d'échelle**Exercice**

Parmi les affirmations suivantes sur la quantification des modèles de deep learning, lesquelles sont correctes ?

☐ **A** La quantification post-entraînement consiste à ré-encoder le modèle existant après calibration sur un set de validation

☐ **B** La quantification-aware training intègre les contraintes de représentation réduite dès la phase d'apprentissage

☐ **C** Tous les types d'architectures réagissent de façon identique à la quantification

☐ **D** Passer de FP32 à INT8 permet de réduire la consommation mémoire tout en accélérant l'inférence

☐ **E** La quantification est uniquement applicable aux modèles de type CNN et ne concerne pas les transformers

Exercice

Associez chaque technique de compression à sa description opérationnelle.

A Quantification-aware training

B Pruning non structuré

C Pruning structuré

D Distillation de connaissances

Suppression de blocs entiers (filtres, neurones, têtes d'attention) permettant des gains significatifs lors du déploiement matériel	Retrait individuel de poids jugés négligeables, indépendamment de leur position dans la couche	Entraînement d'un modèle compact à imiter les sorties probabilistes d'un modèle plus massif	Apprentissage du modèle avec des contraintes de précision numérique réduite intégrées dès l'origine
---	--	---	---

Exercice

Remettez dans le bon ordre les étapes d'une démarche de compression par pruning suivie d'une validation rigoureuse.

A Identifier et supprimer les poids ou blocs les moins influents selon la stratégie de pruning choisie

B Effectuer un ou plusieurs cycles de ré-entraînement pour stabiliser la précision du modèle allégé

C Archiver le modèle d'origine (teacher) et établir un pipeline d'évaluation complet comme référence

D Valider le compromis performance/complexité et décider de conserver ou d'ajuster le niveau de pruning

E Mesurer la perte de précision sur l'ensemble des sous-populations critiques, pas uniquement sur la moyenne globale

Réponse : ____ _

Exercice 11 : Optimisation du graphe et du pipeline

[solution n°3 p. 30]

Exercice

Une équipe fusionne les opérations convolution, activation et batch normalization en une seule opération dans le graphe computationnel de son modèle. Quel est le principal bénéfice attendu de cette transformation ?

Une augmentation de la précision du modèle grâce à une meilleure propagation du gradient

Une réduction de la latence et de la consommation mémoire en limitant les allers-retours entre mémoire et unités de calcul

Une simplification du code source qui facilite la maintenance par les équipes

Une compatibilité automatique avec tous les frameworks de déploiement sans conversion supplémentaire

Exercice

Votre pipeline d'inférence présente des ralentissements inexplicables malgré un modèle déjà optimisé. Quelles actions peuvent réduire ces goulots d'étranglement « hors modèle » ?

A

☐

Adopter des formats de données optimisés comme TFRecord ou Parquet pour accélérer la lecture disque-mémoire

B

☐

Augmenter la profondeur du réseau neuronal pour améliorer la précision

C

☐

Mettre en place un préchargement asynchrone des données pour éviter les temps morts d'alimentation

D

☐

Paralléliser le prétraitement sur plusieurs cœurs CPU

E

☐

Remplacer le modèle par une architecture plus récente sans modifier le pipeline

Exercice 14 : Déploiement et intégration MLOps

[solution n°4 p. 31]

Exercice

Un hôpital souhaite déployer un modèle d'aide au diagnostic sur ses postes internes. Les données patients sont hautement confidentielles et la latence doit rester inférieure à 100 ms. Quelle infrastructure est la plus adaptée ?

Cloud public mutualisé avec autoscaling pour absorber les pics de charge

Déploiement local sur serveurs internes disposant de GPU, sans transit des données vers l'extérieur

Edge device léger sans GPU pour minimiser les coûts d'infrastructure

Architecture hybride avec stockage des données patients sur cloud privé et inférence déportée

Exercice

Dans un pipeline MLOps intégrant l'optimisation computationnelle, quelles pratiques contribuent directement à sécuriser la performance en production ?

A☐

Intégrer des étapes de profilage et de benchmarking automatisés à chaque nouveau push de modèle

B☐

Générer automatiquement des versions optimisées (quantifiées, compilées) et les tester selon les critères métier

C☐

Déployer directement en production le modèle le plus précis sans phase de test multi-plateforme

D☐

Mettre en place des alertes automatiques en cas de régression de performance ou de consommation excessive

E☐

Documenter les retours terrain et capitaliser les recettes d'optimisation réussies pour les équipes futures

VII Exercice : Optimiser un modèle de segmentation d'images médicales pour le déploiement hospitalier

Mise en situation professionnelle

Vous êtes ingénieur·e MLOps au sein de **MedVision**, une entreprise française spécialisée dans les solutions d'imagerie médicale destinées aux établissements hospitaliers publics et privés. Votre équipe a développé un modèle de **segmentation d'organes** basé sur l'architecture **U-Net**, entraîné sur un corpus de **12 000 scanners thoraciques** annotés par des radiologues.

Le modèle atteint un **Dice score de 0.94** sur le jeu de validation, ce qui satisfait pleinement les exigences cliniques. Cependant, son déploiement sur les postes hospitaliers standards pose un problème majeur : le temps d'inférence moyen est de **2,8 secondes par image** sur les GPU de gamme moyenne (NVIDIA T4) équipant les stations de travail des hôpitaux partenaires. Or, le workflow clinique impose un temps de réponse inférieur à **1,1 seconde** pour s'intégrer sans friction dans la chaîne de lecture des radiologues.

Voici les contraintes techniques et opérationnelles du projet :

Paramètre	Valeur actuelle	Cible
Temps d'inférence moyen	2,8 s	< 1,1 s (réduction ≥ 60 %)
Dice score (précision)	0,94	$\geq 0,92$ (perte max 2 points)
Taille du modèle	340 Mo (FP32)	< 120 Mo
RAM GPU disponible	16 Go (NVIDIA T4)	Occupation < 70 %
Résolution d'entrée	512 × 512 pixels	Maintenue
Environnement OS	Ubuntu 20.04, CUDA 11.8	Identique
Connectivité réseau	Variable (parfois instable)	Exécution 100 % locale

Le directeur technique vous demande de piloter l'ensemble de la démarche d'optimisation : du diagnostic de performance initial jusqu'au déploiement stabilisé en production hospitalière. Vous devez garantir la **traçabilité** de chaque modification, la **reproductibilité** des résultats et la **conformité** aux exigences réglementaires du secteur médical (marquage CE, auditabilité des versions).

Votre équipe dispose des outils suivants :

- **PyTorch Profiler** et **NVIDIA Nsight** pour le profilage
- **ONNX Runtime** et **TensorRT** pour l'optimisation du graphe
- **Docker** et **MLflow** pour la containerisation et le versioning
- Un jeu de calibration de **500 images** représentatives pour la quantification

Données complémentaires

Voici un extrait du rapport de profilage préliminaire réalisé par un collègue sur la station de développement (GPU A100) :

Étape du pipeline	Temps moyen (ms)	% du temps total
Chargement image DICOM	120	8 %
Prétraitement (normalisation, resize)	210	14 %
Inférence U-Net (forward pass)	980	65 %
Post-traitement (seuillage, lissage)	90	6 %
Sérialisation résultat	105	7 %
Total	1 505	100 %

Attention : ces mesures ont été réalisées sur GPU A100 (station de développement). Sur le GPU cible T4, le temps d'inférence est multiplié par un facteur estimé de **1,8 à 2x**, ce qui explique les 2,8 s observées en conditions réelles.

Par ailleurs, l'équipe a noté que :

- Le modèle utilise des poids en **FP32** sans aucune optimisation de graphe
- Le prétraitement s'exécute **séquentiellement sur CPU** sans parallélisation
- Le chargement DICOM utilise une bibliothèque non optimisée pour le batch processing
- Aucune fusion d'opérations n'a été appliquée au graphe computationnel

Question 1

[solution n°5 p. 32]

Concevez une méthodologie de profilage structurée pour identifier les goulots d'étranglement du modèle actuel sur l'environnement cible hospitalier. Précisez les outils, les métriques prioritaires et les conditions de mesure à respecter.

Question 2

[solution n°6 p. 33]

Proposez au moins trois techniques complémentaires issues de la compression, de l'optimisation du graphe ou du code, et de l'adaptation au déploiement ciblé pour atteindre l'objectif de réduction de 60 % du temps d'inférence sans dépasser 2 points de perte de Dice score. Justifiez la complémentarité de vos choix.

Question 3

[solution n°7 p. 34]

Définissez un plan d'expérimentation rigoureux pour mesurer l'impact de chaque optimisation sur les performances computationnelles et la précision diagnostique. Précisez le protocole de comparaison, les critères de validation et les garde-fous.

Question 4

[solution n°8 p. 35]

Élaborez une stratégie de déploiement efficace prenant en compte les contraintes matérielles, logicielles et réglementaires de l'environnement hospitalier. Intégrez les dimensions de portabilité, de résilience et de maintenance continue.

Solutions des exercices

Solution n°1

[exercice p. 19]

Exercice

Profiler un modèle IA exclusivement sur une machine de développement haute performance est suffisant pour anticiper les performances en production.

Vrai

✓ Faux



C'est faux. Profiler uniquement sur une machine puissante peut masquer des goulots d'étranglement qui n'apparaîtront qu'en environnement cible réel. Par exemple, un modèle peut afficher une latence acceptable sur une station de développement et être 10 fois plus lent sur une puce ARM embarquée, comme l'illustre le cas de la startup avec ses casques Bluetooth. Le profilage doit toujours être réalisé dans des conditions réalistes ou en simulant la configuration finale.

Exercice

Vous déployez un modèle IA sur un capteur connecté embarqué, sans connexion réseau stable. Quelle combinaison de métriques doit primer dans votre démarche de benchmarking ?

Débit maximal et scalabilité horizontale

✓ Latence, consommation mémoire et autonomie énergétique

Précision globale et coût d'inférence par requête

Taux d'utilisation du batch processing et occupation I/O réseau



Sur un appareil edge embarqué sans connexion stable, les contraintes déterminantes sont la latence (réactivité locale), la consommation mémoire (ressources limitées) et l'autonomie énergétique (alimentation contrainte). Le débit et la scalabilité horizontale sont des métriques cloud. La précision globale seule ne suffit pas à piloter l'optimisation dans ce contexte. L'occupation I/O réseau est peu pertinente en l'absence de connexion stable.

Exercice

Associez chaque outil ou concept de profilage à sa description fonctionnelle principale.

Analyse ligne à ligne des opérations et visualisation de la charge sur chaque composant matériel	Représentation visuelle permettant d'identifier les nœuds critiques qui bloquent la parallélisation	Quantité maximale de données transférable entre mémoire et unités de calcul, dont la saturation crée un goulot matériel incompressible	Ensemble de mesures de référence permettant d'évaluer objectivement les gains ou pertes d'une optimisation
PyTorch Profiler / TensorFlow Profiler	Heatmap ou graphe d'exécution	Bande passante mémoire	Baseline

PyTorch Profiler et TensorFlow Profiler permettent d'analyser les opérations ligne à ligne et de visualiser la charge matérielle. La heatmap ou le graphe d'exécution rendent visibles les nœuds critiques qui bloquent la chaîne. La bande passante mémoire est la limite physique des transferts entre mémoire et calcul : aucune optimisation logicielle ne peut la compenser si elle est saturée. La baseline est le référentiel de mesures qui permet de juger objectivement l'impact de chaque modification.

Solution n°2

[exercice p. 20]

Exercice

Le pruning d'un réseau neuronal dégrade systématiquement sa capacité de généralisation.

Vrai

✓ Faux

C'est faux. Le pruning peut, dans certains cas, améliorer la généralisation en éliminant des poids redondants ou sur-spécialisés. C'est notamment le cas lorsqu'il est combiné à des ré-entraînements successifs qui stabilisent la précision. La dégradation n'est pas systématique : elle survient surtout lorsque le pruning est excessif ou mal calibré, supprimant des poids essentiels. La leçon précise explicitement que le pruning 'améliore parfois la généralisation'.

Exercice

Dans une démarche de distillation de connaissances, qu'est-ce que le modèle « élève » (student) apprend à reproduire en priorité ?

Les étiquettes binaires exactes (hard targets) issues du jeu d'entraînement d'origine

✓ Les sorties probabilistes du modèle teacher (soft targets), qui capturent la logique fine du réseau massif

Les poids internes du modèle teacher, couche par couche

La structure architecturale complète du modèle teacher, réduite par facteur d'échelle



La distillation repose précisément sur le fait que le student n'apprend pas les simples étiquettes binaires (hard targets), mais les sorties probabilistes du teacher (soft targets). Ces soft targets contiennent une information bien plus riche sur les relations entre classes et la logique fine du réseau, ce qui permet au modèle compact d'hériter d'une partie de la robustesse du modèle massif. Copier les poids ou l'architecture du teacher n'est pas l'objet de la distillation.

Exercice

Parmi les affirmations suivantes sur la quantification des modèles de deep learning, lesquelles sont correctes ?

A



La quantification post-entraînement consiste à ré-encoder le modèle existant après calibration sur un set de validation

B



La quantification-aware training intègre les contraintes de représentation réduite dès la phase d'apprentissage

C

Tous les types d'architectures réagissent de façon identique à la quantification

D



Passer de FP32 à INT8 permet de réduire la consommation mémoire tout en accélérant l'inférence

E

La quantification est uniquement applicable aux modèles de type CNN et ne concerne pas les transformers



La quantification post-entraînement et la quantification-aware training sont deux approches distinctes et correctement décrites dans les propositions 1 et 2. Le passage à INT8 ou INT4 réduit effectivement la mémoire et accélère l'inférence (proposition 4). En revanche, les architectures ne réagissent pas toutes de la même façon : certains transformers ou réseaux à attention sont plus sensibles à la quantification que les CNN ou MLP. La quantification n'est pas réservée aux CNN : elle s'applique à divers types de réseaux, même si l'impact varie.

Exercice

Associez chaque technique de compression à sa description opérationnelle.

Suppression de blocs entiers (filtres, neurones, têtes d'attention) permettant des gains significatifs lors du déploiement matériel	Retrait individuel de poids jugés négligeables, indépendamment de leur position dans la couche	Entraînement d'un modèle compact à imiter les sorties probabilistes d'un modèle plus massif	Apprentissage du modèle avec des contraintes de précision numérique réduite intégrées dès l'origine
Pruning structuré	Pruning non structuré	Distillation de connaissances	Quantification-aware training

🔍 Le pruning structuré supprime des blocs entiers (filtres, canaux, têtes), ce qui génère des gains matériels directs. Le pruning non structuré retire des poids individuels proches de zéro, sans considération de leur position. La distillation entraîne un student à reproduire les soft targets d'un teacher massif. La quantification-aware training intègre les contraintes de représentation réduite pendant l'entraînement lui-même, ce qui lui confère une meilleure résilience à la perte de précision par rapport à la quantification post-entraînement.

Exercice

Remettez dans le bon ordre les étapes d'une démarche de compression par pruning suivie d'une validation rigoureuse.

A Archiver le modèle d'origine (teacher) et établir un pipeline d'évaluation complet comme référence

B Identifier et supprimer les poids ou blocs les moins influents selon la stratégie de pruning choisie

C Effectuer un ou plusieurs cycles de ré-entraînement pour stabiliser la précision du modèle allégé

D Mesurer la perte de précision sur l'ensemble des sous-populations critiques, pas uniquement sur la moyenne globale

E Valider le compromis performance/complexité et décider de conserver ou d'ajuster le niveau de pruning

🔍 La démarche commence obligatoirement par l'archivage du modèle de référence, sans quoi aucune comparaison objective n'est possible. On procède ensuite au pruning, puis au ré-entraînement pour compenser la perte induite. L'évaluation doit porter sur toutes les sous-populations critiques (et non la seule moyenne globale, qui peut masquer des dégradations locales importantes). Enfin, la décision de conserver ou d'ajuster le niveau de pruning s'appuie sur l'ensemble de ces mesures.

Solution n°3

Exercice

Une équipe fusionne les opérations convolution, activation et batch normalization en une seule opération dans le graphe computationnel de son modèle. Quel est le principal bénéfice attendu de cette transformation ?

Une augmentation de la précision du modèle grâce à une meilleure propagation du gradient

✓ Une réduction de la latence et de la consommation mémoire en limitant les allers-retours entre mémoire et unités de calcul

Une simplification du code source qui facilite la maintenance par les équipes

Une compatibilité automatique avec tous les frameworks de déploiement sans conversion supplémentaire



La fusion d'opérations regroupe plusieurs calculs successifs en une seule opération optimisée, ce qui réduit les allers-retours en mémoire et les surcoûts liés aux appels séparés. Le résultat direct est une baisse de la latence et de la consommation mémoire. La précision du modèle n'est pas améliorée par cette transformation. La maintenance du code et la compatibilité multi-frameworks sont des considérations distinctes, non directement liées à la fusion d'opérations dans le graphe.

Exercice

Votre pipeline d'inférence présente des ralentissements inexplicables malgré un modèle déjà optimisé. Quelles actions peuvent réduire ces goulots d'étranglement « hors modèle » ?



Adopter des formats de données optimisés comme TFRecord ou Parquet pour accélérer la lecture disque-mémoire

B Augmenter la profondeur du réseau neuronal pour améliorer la précision



Mettre en place un préchargement asynchrone des données pour éviter les temps morts d'alimentation



Paralléliser le prétraitement sur plusieurs cœurs CPU

E Remplacer le modèle par une architecture plus récente sans modifier le pipeline



Les ralentissements hors modèle proviennent souvent de l'accès aux données (I/O) et du prétraitement. Adopter des formats optimisés (TFRecord, Parquet) accélère la lecture disque-mémoire. Le préchargement asynchrone supprime les temps morts entre les lots. La parallélisation du prétraitement répartit la charge sur plusieurs cœurs. En revanche, augmenter la profondeur du réseau aggraverait la charge computationnelle, et remplacer le modèle sans toucher au pipeline ne résout pas les goulots d'étranglement identifiés dans ce dernier.

Solution n°4

[exercice p. 22]

Exercice

Un hôpital souhaite déployer un modèle d'aide au diagnostic sur ses postes internes. Les données patients sont hautement confidentielles et la latence doit rester inférieure à 100 ms. Quelle infrastructure est la plus adaptée ?

Cloud public mutualisé avec autoscaling pour absorber les pics de charge

✓ Déploiement local sur serveurs internes disposant de GPU, sans transit des données vers l'extérieur

Edge device léger sans GPU pour minimiser les coûts d'infrastructure

Architecture hybride avec stockage des données patients sur cloud privé et inférence déportée



Les impératifs de confidentialité (données médicales sensibles) et de faible latence (<100 ms) orientent vers un déploiement local sur des serveurs internes avec GPU. Le cloud public mutualisé expose les données à des risques de confidentialité et peut introduire de la latence réseau. Un edge device sans GPU ne peut pas garantir la latence requise pour un modèle de segmentation complexe. L'architecture hybride avec stockage cloud, même privé, reste risquée sur le plan réglementaire et introduit une dépendance réseau incompatible avec la latence cible.

Exercice

Dans un pipeline MLOps intégrant l'optimisation computationnelle, quelles pratiques contribuent directement à sécuriser la performance en production ?

A



Intégrer des étapes de profilage et de benchmarking automatisés à chaque nouveau push de modèle

B



Générer automatiquement des versions optimisées (quantifiées, compilées) et les tester selon les critères métier

C

Déployer directement en production le modèle le plus précis sans phase de test multi-plateforme

D

Mettre en place des alertes automatiques en cas de régression de performance ou de consommation excessive

E

Documenter les retours terrain et capitaliser les recettes d'optimisation réussies pour les équipes futures



Un pipeline MLOps mature intègre le profilage et le benchmarking automatisés à chaque itération (proposition 1), génère et teste des versions optimisées sans intervention manuelle (proposition 2), et déclenche des alertes en cas de régression (proposition 4). La capitalisation documentée évite de répéter des cycles coûteux de trial & error et facilite la montée en compétence des nouvelles équipes (proposition 5). En revanche, déployer directement en production sans test multi-plateforme est explicitement identifié comme une pratique risquée dans la leçon : même un modèle performant en test unitaire peut révéler des goulots cachés à l'échelle ou sur du hardware moins performant.

[exercice p. 25] **Solution n°5**

Réponse attendue

La méthodologie de profilage doit impérativement être conduite **sur le matériel cible** (GPU NVIDIA T4, Ubuntu 20.04, CUDA 11.8) et non sur la station de développement A100, afin d'obtenir des mesures représentatives des conditions réelles. Vous structurez le diagnostic en trois phases : d'abord, un **profilage global du pipeline** avec PyTorch Profiler pour mesurer la latence de bout en bout et identifier la répartition du temps entre chargement, prétraitement, inférence et post-traitement. Ensuite, un **profilage granulaire du forward pass** avec NVIDIA Nsight pour cartographier les opérations les plus coûteuses couche par couche (convolutions, skip connections, upsampling) et détecter les transferts mémoire CPU-GPU inutiles. Enfin, une **analyse mémoire** mesurant l'occupation VRAM pic, la bande passante mémoire utilisée et les éventuels débordements vers la RAM système. Chaque mesure est répétée sur un lot de **50 images représentatives** avec calcul de la moyenne et de l'écart-type, et les résultats sont consignés dans un tableau de bord servant de **baseline** pour évaluer chaque optimisation ultérieure.

Éléments de solution

- **Outils à mobiliser** : PyTorch Profiler pour la vue d'ensemble du pipeline, NVIDIA Nsight pour l'analyse GPU kernel par kernel, nvidia-smi pour le monitoring VRAM en temps réel
- **Métriques prioritaires** : latence par étape (ms), occupation VRAM pic (Go), bande passante mémoire utilisée (%), taux d'utilisation GPU (%), débit (images/seconde)
- **Condition critique** : toujours profiler sur le **GPU cible T4** et non sur la station de développement — le rapport préliminaire montre un facteur 1,8 à 2x entre A100 et T4

- **Structuration en phases :**
 - Phase 1 : profilage macro (pipeline complet, identification des étapes dominantes)
 - Phase 2 : profilage micro (opérations internes au forward pass, heatmap des kernels)
 - Phase 3 : analyse mémoire et I/O (transferts CPU-GPU, accès disque, cache)
- **Baseline :** consigner toutes les métriques dans un tableau de référence versionné, qui servira de point de comparaison pour chaque itération d'optimisation
- **Rappel méthodologique :** le profilage préliminaire révèle que l'inférence représente **65 %** du temps total — c'est le levier prioritaire, mais le prétraitement séquentiel (14 %) constitue un second goulot significatif à ne pas négliger

[exercice p. 25] **Solution n°6**

Réponse attendue

Pour atteindre l'objectif, vous combinez trois leviers complémentaires agissant à des niveaux différents du pipeline. **Premier levier — quantification mixte FP16/INT8** : vous convertissez les poids du modèle de FP32 vers FP16 pour les couches peu sensibles et INT8 pour les convolutions les plus lourdes, en utilisant le jeu de calibration de 500 images. Cette opération réduit la taille du modèle d'environ **340 Mo à 85-100 Mo** et accélère l'inférence sur T4 grâce aux Tensor Cores, avec une perte de Dice score généralement inférieure à 0,5 point. **Deuxième levier — optimisation du graphe computationnel via TensorRT** : vous exportez le modèle au format ONNX puis compilez avec TensorRT, qui applique automatiquement la fusion d'opérations (convolution + batch norm + ReLU), l'élimination des nœuds redondants et l'optimisation mémoire. Ce seul levier peut réduire la latence d'inférence de **30 à 50 %**. **Troisième levier — parallélisation du prétraitement et chargement asynchrone** : vous remplacez le chargement DICOM séquentiel par une bibliothèque optimisée avec préchargement asynchrone sur plusieurs cœurs CPU, pendant que le GPU traite l'image précédente. Cette optimisation du pipeline hors-modèle élimine les temps morts et réduit la contribution du prétraitement de **210 ms à environ 60-80 ms**. La complémentarité est essentielle : la quantification agit sur la **précision numérique**, TensorRT sur la **structure du graphe**, et la parallélisation sur le **pipeline de données** — trois couches distinctes dont les gains se cumulent.

Éléments de solution

- **Quantification (compression) :**
 - Passage FP32 → FP16/INT8 via quantification post-entraînement avec calibration
 - Gain attendu : réduction taille modèle de ~70 %, accélération inférence de 20-40 % sur T4
 - Risque : surveiller la perte de précision sur les structures anatomiques fines (petits organes)
- **Optimisation du graphe (TensorRT) :**
 - Export ONNX → compilation TensorRT avec fusion d'opérations automatique
 - Gain attendu : réduction latence forward pass de 30-50 %
 - Les skip connections du U-Net bénéficient particulièrement de l'optimisation mémoire
- **Parallélisation du pipeline (code) :**
 - Préchargement asynchrone des images DICOM, prétraitement multi-thread sur CPU
 - Gain attendu : réduction du temps hors-inférence de ~50 %

- **Techniques complémentaires envisageables :**
 - **Pruning structuré** des filtres de convolution les moins contributifs dans l'encodeur U-Net
 - **Distillation** vers un U-Net plus étroit si les trois premiers leviers ne suffisent pas
- **Principe de complémentarité :** chaque technique agit sur une couche différente (précision numérique / structure du graphe / pipeline de données), ce qui permet de **cumuler les gains** sans interférence
- **Rappel :** toujours valider le Dice score après chaque optimisation sur le jeu de validation complet, et pas uniquement sur la moyenne globale — vérifier les performances par classe d'organe

[exercice p. 25] **Solution n°7**

Réponse attendue

Le plan d'expérimentation repose sur un protocole de **comparaison incrémentale** : chaque optimisation est appliquée individuellement puis en combinaison, et ses résultats sont systématiquement comparés à la baseline établie lors du profilage initial. Vous définissez un **jeu de test fixe de 1 000 images** non utilisées pour la calibration, stratifié par type d'organe et par niveau de difficulté (cas standards et cas limites). Pour chaque configuration testée, vous mesurez simultanément les **métriques computationnelles** (latence moyenne, latence P95, occupation VRAM pic, taille du modèle) et les **métriques de précision** (Dice score global, Dice score par classe d'organe, taux de faux positifs sur les structures critiques). Les critères de validation sont binaires : la configuration est retenue si la latence est **< 1,1 s** et le Dice score **≥ 0,92** sur toutes les classes. Comme garde-fou, vous instaurez un **test de non-régression automatisé** intégré au pipeline CI/CD via MLflow : tout commit générant une régression au-delà des seuils déclenche une alerte bloquante. Enfin, chaque expérimentation est documentée avec ses hyperparamètres, sa configuration matérielle et ses résultats dans un registre versionné, garantissant la **reproductibilité** et l'**auditabilité** exigées par le cadre réglementaire médical.

Éléments de solution

- **Protocole incrémental** : tester chaque optimisation isolément (quantification seule, TensorRT seul, pipeline seul), puis les combinaisons, pour isoler la contribution de chaque levier
- **Jeu de test** : 1 000 images fixes, stratifiées par organe et par difficulté, distinctes du jeu de calibration
- **Métriques computationnelles** : latence moyenne, latence P95 (percentile 95 pour détecter les pics), VRAM pic, taille modèle, débit (images/s)
- **Métriques de précision** : Dice score global et **par classe d'organe** — ne pas se limiter à la moyenne globale qui peut masquer une dégradation sur les petites structures
- **Critères de validation binaires** :
 - Latence **< 1,1 s** sur GPU T4
 - Dice score **≥ 0,92** sur chaque classe d'organe
- **Garde-fous** :
 - Tests de non-régression automatisés dans le pipeline CI/CD
 - Alertes bloquantes en cas de dépassement des seuils
 - Revue croisée des résultats entre data scientists et radiologues référents

- **Traçabilité** : chaque expérimentation versionnée dans MLflow (modèle, hyperparamètres, métriques, configuration matérielle) pour garantir la reproductibilité et répondre aux exigences d'audit réglementaire (marquage CE)

[exercice p. 25] **Solution n°8**

Réponse attendue

La stratégie de déploiement s'articule autour de trois piliers adaptés au contexte hospitalier.

Premier pilier — containerisation et portabilité : vous empaquetez le modèle optimisé, son runtime TensorRT et l'ensemble du pipeline de prétraitement dans un **conteneur Docker** versionné, garantissant un environnement d'exécution identique sur tous les postes hospitaliers, indépendamment des variations de configuration locale. L'image Docker inclut les drivers CUDA compatibles et est testée sur la configuration T4 de référence.

Deuxième pilier — exécution 100 % locale et résilience : compte tenu de la connectivité réseau instable dans certains établissements, le modèle fonctionne entièrement en local sans dépendance cloud. Un mécanisme de **fallback** est prévu : si le GPU rencontre une erreur, une version CPU allégée (quantifiée INT8, latence dégradée mais fonctionnelle) prend le relais pour assurer la continuité de service. Les résultats sont mis en cache localement et synchronisés vers le serveur central dès que la connexion est rétablie.

Troisième pilier — maintenance et conformité : chaque version déployée est tracée dans **MLflow** avec son numéro de version, ses métriques de validation et son hash de modèle. Les mises à jour sont déployées via un pipeline CI/CD qui exécute automatiquement les tests de non-régression avant toute diffusion. Un tableau de bord centralisé permet de monitorer la latence et le taux d'erreur en temps réel sur l'ensemble du parc, et un mécanisme de **rollback automatique** restaure la version précédente en cas de régression détectée. Cette architecture répond aux exigences d'**auditabilité** et de **traçabilité** imposées par le marquage CE des dispositifs médicaux.

Éléments de solution

- **Containerisation** : Docker avec image incluant le modèle TensorRT, les dépendances CUDA, le pipeline de prétraitement — garantit la reproductibilité sur tout le parc
- **Exécution locale obligatoire** : aucune dépendance réseau pour l'inférence, conformément aux contraintes de connectivité et de confidentialité des données patients
- **Mécanisme de fallback** : version CPU de secours (INT8) activée automatiquement en cas de défaillance GPU, assurant la continuité de service même en mode dégradé
- **Versioning et traçabilité** : MLflow pour le suivi de chaque version déployée (modèle, métriques, configuration), indispensable pour l'audit réglementaire
- **Pipeline CI/CD** : tests de non-régression automatisés avant chaque déploiement, rollback automatique si les seuils de latence ou de précision sont dépassés
- **Monitoring en production** : tableau de bord centralisé surveillant latence, VRAM, taux d'erreur sur chaque poste — détection proactive des dérives
- **Conformité réglementaire** :
 - Traçabilité complète des versions pour le marquage CE
 - Confidentialité des données patients : aucune donnée médicale ne transite hors de l'établissement
 - Documentation des modifications pour les audits qualité
- **Collaboration inter-métiers** : impliquer les radiologues dans la validation clinique de chaque mise à jour, et les équipes IT hospitalières dans les tests d'intégration sur le parc existant